

A Project Report On

Fraud detection

Submitted in partial fulfillment of the requirement for the
award of the degree

MASTER OF SCIENCE (DATA SCIENCE)

from

Marwadi University

Academic Year 2024 – 25

Jayesh Parmar (92300567011)

Aaditya Patadiya (92300567004)

Internal Guide

Prof. Vinod kumar Pal



Marwadi
University
Marwadi Chandarana Group



Marwadi University

Rajkot-Morbi Road, At & PO : Gauridad, Rajkot 360 003. Gujarat. India.



Marwadi
University
Marwadi Chandarana Group



Faculty of Computer Applications (FoCA)

Certificate

This is to certify that the project work entitled
FRUAD DETECTION
submitted in partial fulfillment of the requirement for
the award of the degree of
Master of Science (Data Science)
of the
Marwadi University

is a result of the bonafide work carried out by

Jayesh Parmar (92300567011)
Aaditya Patadiya (92300567004)
during the academic year 2024 – 2025

Faculty Guide

HOD

Dean

DECLARATION

We hereby declare that this project work entitled Fraud detection is a record done by us.

We also declare that the matter embodied in this project is genuine work done by us and has not been submitted whether to this University or to any other University / Institute for the fulfillment of the requirement of any course of study.

Place: Rajkot

Date: 22/04/2025

Jayesh Parmar (92300567011)
Aaditya Patadiya (92300567004)

Signature: _____
Signature: _____



Marwadi
University
Marwadi Chandarana Group



Date: 21/04/2025

PROJECT COMPLETION CERTIFICATE

This is to certify that **AADITYA HARESHBHAI PATADIYA (92300567004)** of **M.SC.(DATA SCIENCE) – 4** have successfully completed the User Defined Project titled "**Fraud detection**" at **Marwadi University** during the period from 01/01/2025 to 21/04/2025.

The project was carried out with dedication, professionalism, and a high standard of quality. We appreciate the efforts and contributions made by [student name], which played a significant role in the successful completion of the project.

We wish them all the best for future endeavors.

Issued on: 21/04/2025

Dr. R. Sridaran,
Dean,
Faculty of Computer Applications
Marwadi University, Rajkot



Established under Gujarat Private Universities Act No. 9 of 2016 | Recognized under section 2f of UGC Act 1956

Rajkot - Morbi Highway, Rajkot, Gujarat, India 360003 | info@marwadiuniversity.ac.in
www.marwadiuniversity.ac.in | +91 89800 30090 | +91 281 7123456



Marwadi
University
Marwadi Chandarana Group



Date: 21/04/2025

PROJECT COMPLETION CERTIFICATE

This is to certify that **JAYESHKUMAR DINESHBHAI PARMAR (92300567011)** of **M.SC.(DATA SCIENCE) – 4** have successfully completed the User Defined Project titled "**Fraud detection**" at **Marwadi University** during the period from 01/01/2025 to 21/04/2025.

The project was carried out with dedication, professionalism, and a high standard of quality. We appreciate the efforts and contributions made by [student name], which played a significant role in the successful completion of the project.

We wish them all the best for future endeavors.

Issued on: 21/04/2025

Dr. R. Sridaran,
Dean,
Faculty of Computer Applications
Marwadi University, Rajkot



Established under Gujarat Private Universities Act No. 9 of 2016 | Recognized under section 2f of UGC Act 1956

Rajkot - Morbi Highway, Rajkot, Gujarat, India 360003 | info@marwadiuniversity.ac.in
www.marwadiuniversity.ac.in | +91 89800 30090 | +91 281 7123456

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us. No serious and lasting achievement or success one can ever achieve without the help of friendly guidance and co-operation of so many people involved in the work.

We are very thankful to our guide **Prof. Vinod kumar Pal** , the person who makes us to follow the right steps during our project work. We express our deep sense of gratitude to for his /her guidance, suggestions and expertise at every stage. A part from that his/her valuable and expertise suggestion during documentation of our report indeed help us a lot.

Thanks to our friends who have been a source of inspiration and motivation that helped us during our project work. We are heartily thankful to our external guide

Prof. Vinod kumar Pal for providing us an opportunity to work over this report and for their endless and great support. We are also thankful to our Head **Dr. Sunil Bajaja** and Dean **Dr. R. Sridaran** for their support as and when required. we would like to extend our gratitude towards all who directly or indirectly supported and helped us to accomplish our project work.

Jayesh Parmar (92300567011)
Aaditya Patadiya (92300567004)

Signature: _____
Signature: _____

CONTENTS

Chapters	Particulars	Page No.
1	Introduction 1.1. Objective of the New System 1.2. Problem Definition 1.3. Core Components 1.4. Project Profile 1.5. Assumptions and Constraints 1.6. Advantages and Limitations of the Proposed System	2-5
2	Requirement Determination & Analysis 2.1. Requirement Determination 2.2. Targeted Users 2.3. Tool details (Python / PowerBI/ Tableau) 2.4. Library description (Details on various libraries / packages used)	6-8
3	System Design 3.1. Flowchart / Algorithm with steps 3.2. Dataset Design 3.3. Details on preprocessing steps applied	9-10
4	Development 4.1 Script details / Source code 4.2. Screen Shots / UI Design of simulation (if applicable) 4.3. Test reports	11-28
5	Proposed Enhancements	29
6	Conclusion	30
7	Bibliography	31

1. Introduction

1.1 Objective of the New System

- Detect fraudulent transactions/activities in real-time.
- Reduce false positives and improve accuracy using machine learning.
- Enhance security and minimize financial losses.

1.2 Problem Definition

- Fraud causes significant financial losses across industries (banking, e-commerce, insurance).
- Traditional rule-based systems fail to adapt to evolving fraud patterns.
- Need for an automated, scalable, and data-driven solution.

1.3 Core Components

1. Problem Definition

- Objective: Understand the problem trying to solve, the business goal, and how machine learning can be applied.
- Key Tasks:
 - Define the problem type: Is it a classification, regression, clustering, or other type of problem?
 - Understand the input and output variables.
 - Determine evaluation metrics (e.g., accuracy, F1-score, RMSE).
- Output: Clear understanding of the problem, data required, and success metrics.

2. Data Collection

- Objective: Gather the data required to train and evaluate the model.
- Key Tasks:
 - Collect data from various sources (databases, APIs, files, etc.).
 - Ensure that the data is relevant to the problem being solved.
- Output: Raw dataset ready for preprocessing.

3. Data Exploration (Exploratory Data Analysis - EDA)

- Objective: Explore and understand the structure, distribution, and relationships within the data.
- Key Tasks:
 - Visualize the data using histograms, box plots, scatter plots, etc.
 - Check for missing values, outliers, and duplicate data.
 - Calculate summary statistics (mean, median, standard deviation).
 - Understand data distributions and feature correlations.
- Tools: Pandas, Matplotlib, Seaborn.
- Output: Insights into the data's structure and potential issues that need to be addressed during preprocessing.

4. Data Preprocessing

- Objective: Clean and transform the data to make it ready for machine learning models.
- Key Tasks:
 - Handle missing data (e.g., imputation or removal).
 - Encode categorical variables (e.g., label encoding, one-hot encoding).
 - Scale/normalize numerical features (e.g., MinMax scaling, standardization).
 - Address class imbalance using techniques like SMOTE or undersampling/oversampling.

- Feature engineering (create new features or transform existing ones).
- Tools: Scikit-learn (SimpleImputer, LabelEncoder, StandardScaler, etc.), SMOTE from imbalanced-learn, etc.
- Output: Cleaned, preprocessed dataset ready for training.

5. Feature Selection

- Objective: Select the most relevant features for the model to improve performance and reduce complexity.
- Key Tasks:
 - Remove highly correlated features (check using VIF).
 - Use statistical methods (e.g., SelectKBest, RFE) to select important features.
 - Use model-based methods (e.g., feature importance from random forests).
- Tools: Scikit-learn (RandomForestClassifier for feature importance).
- Output: Reduced feature set that retains key information.

6. Model Selection

- Objective: Choose the machine learning model(s) that will best solve the problem.
- Key Tasks:
 - Choose models based on the problem type (e.g., LogisticRegression, RandomForest, SVM, etc.).
 - If unsure, try multiple models to compare performance.
- Tools: Scikit-learn, AutoML libraries like AutoGluon, etc.
- Output: A chosen model ready to be trained.

7. Model Training

- Objective: Train the model using the training dataset.
- Key Tasks:
 - Split the dataset into training and testing sets.
 - Train the model using the training data and evaluate on a validation set.
 - Tune hyperparameters if necessary (using grid search or random search).
- Tools: Scikit-learn (train_test_split, GridSearchCV, RandomizedSearchCV) etc.
- Output: Trained model with optimal hyperparameters.

8. Model Evaluation

- Objective: Evaluate the trained model's performance on the test set to understand how well it generalizes to new data.
- Key Tasks:
 - Evaluate the model using relevant metrics such as accuracy, precision, recall, F1-score, or AUC-ROC.
 - Analyze confusion matrix, classification report, and ROC curve for deeper understanding.
 - Compare different models' performances if necessary.
- Tools: Scikit-learn (accuracy_score, confusion_matrix, classification_report, roc_auc_score).
- Output: Performance metrics and an evaluation of the model's strengths and weaknesses.

9. Hyperparameter Tuning

- Objective: Optimize the model by adjusting its hyperparameters to improve performance.
- Key Tasks:
 - Use methods like Grid Search, Random Search, or Bayesian Optimization to find the best hyperparameters.
 - Experiment with different models' hyperparameters (e.g., number of trees for Random Forest, learning rate for Gradient Boosting).
- Tools: Scikit-learn (GridSearchCV, RandomizedSearchCV), Optuna, Hyperopt.
- Output: Best-performing model with optimized hyperparameters.

10. Model Validation (Cross-Validation)

- Objective: Validate the model's performance more robustly by splitting the dataset into multiple training and validation sets.
- Key Tasks:
 - Use k-fold cross-validation to ensure that the model is not overfitting to a particular subset of the data.
 - Calculate performance metrics across different folds to get a better estimate of model generalization.
- Tools: Scikit-learn (cross_val_score, KFold, StratifiedKFold).
- Output: Cross-validation results indicating model robustness and generalizability.

11. Model Interpretation

- Objective: Understand how the model is making predictions to ensure transparency, fairness, and reliability.
- Key Tasks:
 - Use SHAP or LIME to explain individual predictions.
 - Check feature importance to understand which features are influencing predictions the most.
 - Ensure the model's decisions align with domain knowledge and make sense.
- Tools: SHAP, LIME, Scikit-learn (feature importance).
- Output: Model explanations and insights into the decision-making process.

12. Model Deployment

- Objective: Deploy the trained model so that it can be used for real-time predictions in a production environment.
- Key Tasks:
 - Serialize the trained model using joblib or pickle.
 - Develop an API (using Flask, FastAPI, etc.) that serves the model for prediction requests.
 - Deploy the API to a cloud platform or on-premises infrastructure.
- Tools: Flask, FastAPI, Docker, AWS, Azure, GCP.
- Output: A deployed model API ready to be used for real-time predictions.

13. Model Monitoring and Maintenance

- Objective: Continuously monitor model performance in production and retrain it when necessary.
- Key Tasks:
 - Track model performance over time to identify any degradation (e.g., due to data drift).
 - Set up monitoring dashboards to track metrics like latency, throughput, and prediction accuracy.
 - Retrain the model as needed based on new data or performance drops.
- Tools: Prometheus, Grafana, MLflow, ModelDB, Cloud Monitoring Tools.
- Output: Ongoing monitoring and maintenance of the model to ensure it remains accurate and relevant.

1.4 Project Profile

- Domain: Financial Security / Cybersecurity
- Tools: Python (Pandas,matplotlib,numpy,seborn, Scikit-learn), PowerBI
- Dataset: fraud-detection-dataset (e.g., Kaggle's Fraud Detection dataset).

```
LOAD DATASET

data = pd.read_csv("../fraud detection project/Fraud1.csv")
# Randomly sample 10% of the data (you can adjust the fraction)
# data = dataset.sample(frac=0.10, random_state=42)

# Display the sampled data
data
```

[2] ✓ 0.3s

	step	type	amount	nameOrig	oldbalanceOrig	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.00	0.00	0	0
1	1	PAYMENT	1064.28	C1666544295	21249.0	19384.72	M2044282225	0.00	0.00	0	0
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.00	0.00	1	0
3	1	CASH OUT	181.00	C840083671	181.0	0.00	C38997010	21182.00	0.00	1	0
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.00	0.00	0	0
...	...	...	...	...	...	...	...	...	...	...	...
64437	9	CASH_IN	348562.36	C729218685	17800000.0	18200000.00	C16685882	2208457.20	4193397.68	0	0
64438	9	CASH_IN	88337.55	C418473964	18200000.0	18300000.00	C170954749	1106020.06	515073.65	0	0
64439	9	CASH_IN	54499.70	C1021017317	18300000.0	18300000.00	C202064972	1175979.18	1274808.90	0	0
64440	9	CASH_IN	267077.11	C1068253472	18300000.0	18600000.00	C1847868680	949402.83	219057.59	0	0
64441	9	CASH_IN	637034.74	C599119294	18600000.0	19200000.00	C948238125	1220912.12	34315.21	0	0

64442 rows x 11 columns

1.5 Assumptions and Constraints

- Assumes labeled data (fraud/non-fraud).
- Limited by dataset imbalance (fraud cases are rare).
- Real-time detection depends on infrastructure.

1.6 Advantages and Limitations

Advantages	Limitations
High accuracy with ML models.	Requires large, labeled datasets.
Scalable for real-time processing.	False positives may inconvenience users.
Adapts to new fraud patterns.	Computational cost for complex models.

2. Requirement Determination & Analysis

2.1 Requirement Determination

1. Hardware Requirement:

Processor	Minimum Intel Core i3
Hard Disk	Minimum 5 GB
Ram	Minimum 4 GB

2. Software Requirement:

Operating System	Windows 8, 10, 11 (64 bit)
Data	From Kaggle website
Software	Jupyter notebook, Python

2.2 Targeted Users

- Banks: Fraud analysts, risk managers.
- E-commerce: Payment processors.
- End-users: Transparent fraud notifications.

2.3 Tool Details

- Python: Pandas (data cleaning), Scikit-learn (modeling), Imbalanced-learn (handling class imbalance).
- PowerBI: Dashboards for fraud trends.

2.4 Library Description

1. NumPy (np):

- Description: A core library for numerical computing in Python, providing support for arrays, matrices, and many mathematical functions to operate on these data structures. It is especially useful for handling large datasets efficiently.

- Common Uses: Mathematical and logical operations, random number generation, linear algebra, Fourier transforms, etc.

2. Pandas (pd):

- Description: A powerful data manipulation and analysis library. It provides data structures (e.g., DataFrame and Series) for handling structured data, including tools for reading, writing, and manipulating datasets.
- Common Uses: Data cleaning, preparation, and manipulation (e.g., filtering, grouping, merging, pivoting), time series analysis, and handling missing data.

3. Matplotlib (plt):

- Description: A plotting library for creating static, animated, and interactive visualizations in Python.
- Common Uses: Creating line plots, histograms, scatter plots, bar plots, and more.

4. Seaborn (sns):

- Description: A data visualization library based on Matplotlib that provides a high-level interface for drawing attractive and informative statistical graphics.
- Common Uses: Statistical data visualization, heatmaps, pair plots, and distribution plots with better aesthetics than Matplotlib.

5. LabelEncoder from Scikit-learn:

- Description: A utility for encoding categorical labels into numerical values (e.g., converting "apple" → 0, "banana" → 1).
- Common Uses: Converting categorical variables into a format suitable for machine learning models.

6. variance_inflation_factor from Statsmodels:

- Description: A function for calculating the Variance Inflation Factor (VIF), which helps in detecting multicollinearity in regression models.
- Common Uses: To check for correlations among independent variables and identify which variables cause multicollinearity.

7. SMOTE from imbalanced-learn:

- Description: The Synthetic Minority Over-sampling Technique (SMOTE) is used to generate synthetic examples to balance imbalanced datasets.
- Common Uses: Oversampling the minority class in imbalanced datasets to improve the performance of machine learning models.

8. shuffle from Scikit-learn:

- Description: A function that shuffles a dataset or array, usually used before splitting data into training and testing sets.
- Common Uses: Randomizing the order of dataset rows for cross-validation or splitting.

9. SimpleImputer from Scikit-learn:

- Description: A utility for handling missing values in datasets by replacing them with statistical measures such as mean, median, or the most frequent value.
- Common Uses: Imputing missing values in datasets before feeding them into machine learning algorithms.

10. StandardScaler from Scikit-learn:

- Description: A standardization tool that scales features so they have a mean of 0 and a

standard deviation of 1.

- Common Uses: Normalizing data to ensure fair comparison between features and enhance model performance, especially for algorithms sensitive to feature scaling.

11. `train_test_split` from Scikit-learn:

- Description: A utility function to split datasets into training and testing sets, ensuring random sampling of the data.
- Common Uses: Splitting data to train and evaluate machine learning models.

12. `RandomForestClassifier`, `LogisticRegression`, `KNeighborsClassifier`, `SVC`, `DecisionTreeClassifier` from Scikit-learn:

- Description: These are different machine learning algorithms for classification tasks:
 - `RandomForestClassifier`: An ensemble method using multiple decision trees to improve accuracy and reduce overfitting.
 - `LogisticRegression`: A linear model for binary classification, predicting probabilities using the logistic function.
 - `KNeighborsClassifier`: A non-parametric method that classifies data based on the majority class among the k-nearest neighbors.
 - `SVC` (Support Vector Classifier): A model that seeks to find the hyperplane that best separates classes in a high-dimensional space.
 - `DecisionTreeClassifier`: A non-linear model that splits data recursively based on features to classify data points.
- Common Uses: Building and evaluating predictive models for classification problems.

13. `TabularDataset`, `TabularPredictor` from AutoGluon:

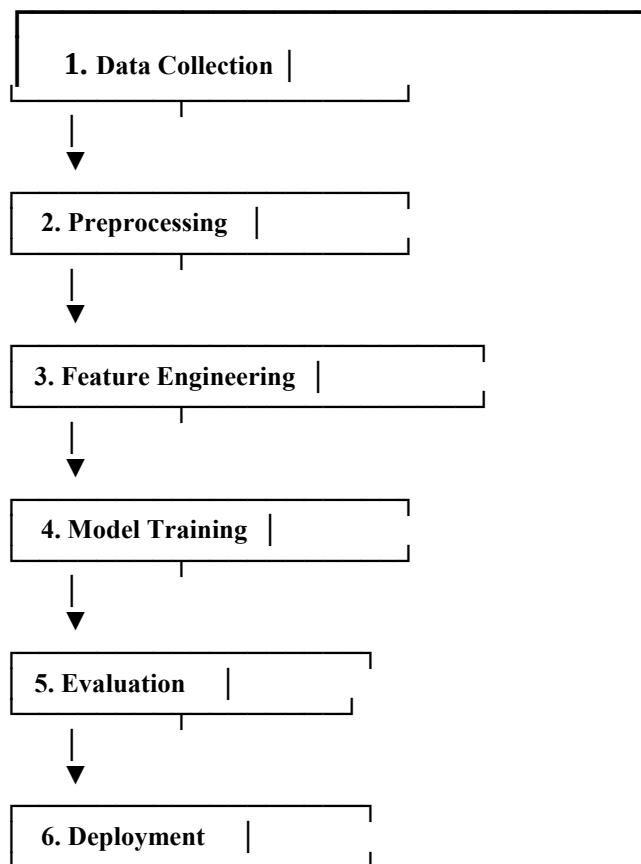
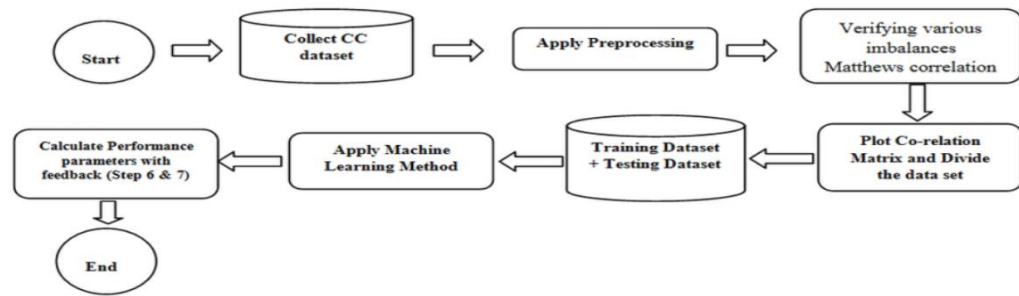
- Description: AutoGluon is an open-source AutoML toolkit for automating the machine learning workflow. `TabularDataset` is used to load tabular data, and `TabularPredictor` is used to train and evaluate models automatically.
- Common Uses: Building machine learning models with minimal code and effort, especially for tabular data.

14. `precision_recall_curve`, `average_precision_score`, `accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_curve`, `roc_auc_score`, `precision_recall_curve`, `auc` from Scikit-learn:

- Description: These are various metrics used for evaluating the performance of classification models.
 - Precision, Recall, F1-Score: Measures of classification performance, focusing on the trade-offs between true positives, false positives, and false negatives.
 - Accuracy: The proportion of correctly classified samples.
 - ROC Curve, AUC: A graph of the true positive rate against the false positive rate, with AUC providing a summary of the model's ability to distinguish between classes.
 - Precision-Recall Curve: A curve that plots precision against recall, useful when dealing with imbalanced datasets.
 - Average Precision Score: The area under the precision-recall curve, summarizing the model's performance in one number.

3. System Design

3.1 Flowchart and workflow



3.2 Dataset Design

- Features: 'step', 'type', 'amount', 'nameOrig', 'oldbalanceOrg', 'newbalanceOrig', 'nameDest', 'oldbalanceDest', 'newbalanceDest', 'isFraud', 'isFlaggedFraud'.
- Shape of the dataset 64442 rows × 11 columns.
- Target Variable: Binary (0: isnotFraud, 1: isFraud).

3.3 Preprocessing Steps

- **Handle missing values (mean/median imputation).**
- **Feature selection and Feature scaling.**

Feature Selection

61. Encoding Categorical Features

```
# identify categorical columns
categorical_cols = ['sender', 'receiver', 'type'] # Replace with actual categorical columns

# apply Label Encoding
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le
```

[70] ✓ 0.3s

62. Split the dataset into features (X) and target (y).

▷ ~

```
x = data.drop(columns=['isFraud'])
y = data['isFraud']
```

[71] ✓ 0.0s

Feature Scaling:

[+ Code](#)[+ Markdown](#)

```
from sklearn.preprocessing import StandardScaler

# Define numerical columns to scale
numerical_columns = ['balanceChangeOrig', 'amount', 'amount_normalized', 'oldbalanceDest',
                    'newbalanceDest', 'type_PAYMENT', 'isZeroBalanceOrig',
                    'balanceChangeDest', 'sender', 'receiver', 'oldbalanceOrig',
                    'type_TRANSFER', 'type_encoded', 'step', 'isLargeBalanceChangeDest',
                    'type']

scaler = StandardScaler()

# Apply scaling
data[numerical_columns] = scaler.fit_transform(data[numerical_columns])
```

- SMOTE for balancing classes.

Address Class Imbalance:

```
from imblearn.over_sampling import SMOTE
from sklearn.utils import shuffle

# Resampling with SMOTE (Synthetic Minority Over-sampling Technique)
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_selected, y)

# Shuffle the dataset after resampling
X_resampled, y_resampled = shuffle(X_resampled, y_resampled, random_state=42)
```

✓ 0.1s

4. Development

4.1 Script Details

Import necessary libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

#from ydata_profiling import ProfileReport
from sklearn.preprocessing import LabelEncoder
from statsmodels.stats.outliers_influence import variance_inflation_factor
from imblearn.over_sampling import SMOTE
from sklearn.utils import shuffle
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from autogluon.tabular import TabularDataset, TabularPredictor
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import average_precision_score
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import roc_curve, roc_auc_score, precision_recall_curve, auc
```

LOAD DATASET

```
data = pd.read_csv("F:/fraud detection project/Fraud1.csv")
# Randomly sample 10% of the data (you can adjust the fraction)
# data = dataset.sample(frac=0.10, random_state=42)
```

```
# Display the sampled data
data
```

step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.00	0.00	0
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.00	0.00	0
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.00	0.00	1
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010	21182.00	0.00	1
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.00	0.00	0
...	...	...	...	...	...	...	...	...	...	...
64437	9	CASH_IN	348562.36	C729218685	17800000.0	18200000.00	C16685882	2208457.20	4193397.68	0
64438	9	CASH_IN	88337.55	C418473964	18200000.0	18300000.00	C170954749	1106020.06	515073.65	0
64439	9	CASH_IN	54499.70	C1021017317	18300000.0	18300000.00	C202064972	1175979.18	1274808.90	0
64440	9	CASH_IN	267077.11	C1068253472	18300000.0	18600000.00	C1847868680	949402.83	219057.59	0
64441	9	CASH_IN	637034.74	C599119294	18600000.0	19200000.00	C948238125	1220912.12	34315.21	0

64442 rows x 11 columns

1. Display the first 5 rows of the dataset.

+ Code

+ Markdown

```
data.head()
```

step	type	amount	nameOrig	oldbalanceOrg	newbalanceOrig	nameDest	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
0	1	PAYMENT	9839.64	C1231006815	170136.0	160296.36	M1979787155	0.0	0.0	0
1	1	PAYMENT	1864.28	C1666544295	21249.0	19384.72	M2044282225	0.0	0.0	0
2	1	TRANSFER	181.00	C1305486145	181.0	0.00	C553264065	0.0	0.0	1
3	1	CASH_OUT	181.00	C840083671	181.0	0.00	C38997010	21182.0	0.0	1
4	1	PAYMENT	11668.14	C2048537720	41554.0	29885.86	M1230701703	0.0	0.0	0

6. Get summary statistics for numerical columns.

```
data.describe()
```

	step	amount	oldbalanceOrig	newbalanceOrig	oldbalanceDest	newbalanceDest	isFraud	isFlaggedFraud
count	64442.000000	6.444200e+04	6.444200e+04	6.444200e+04	6.444200e+04	6.444200e+04	64442.000000	64442.0
mean	7.800317	1.626658e+05	8.153795e+05	8.306013e+05	8.440220e+05	1.177096e+06	0.001629	0.0
std	1.922896	3.303516e+05	2.484668e+06	2.522777e+06	2.401390e+06	2.883064e+06	0.040333	0.0
min	1.000000	6.300000e-01	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000	0.0
25%	8.000000	8.650825e+03	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00	0.000000	0.0
50%	8.000000	4.108988e+04	1.931900e+04	0.000000e+00	1.042500e+04	1.120136e+04	0.000000	0.0
75%	9.000000	1.971248e+05	1.551683e+05	1.772839e+05	4.947040e+05	9.399117e+05	0.000000	0.0
max	9.000000	1.000000e+07	2.850000e+07	2.860000e+07	3.130000e+07	3.200000e+07	1.000000	0.0

4. Get the column names and their data types.

```
data.dtypes
```

```
step          int64
type          object
amount        float64
nameOrig      object
oldbalanceOrg float64
newbalanceOrig float64
nameDest      object
oldbalanceDest float64
newbalanceDest float64
isFraud       int64
isFlaggedFraud int64
dtype: object
```

Feature Selection

61. Encoding Categorical Features

```
# identify categorical columns
categorical_cols = ['sender', 'receiver', 'type'] # Replace with actual categorical columns

# apply Label Encoding
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    data[col] = le.fit_transform(data[col])
    label_encoders[col] = le
```

[70] ✓ 0.3s

62. Split the dataset into features (X) and target (y).

```
x = data.drop(columns=['isFraud'])
y = data['isFraud']
```

[71] ✓ 0.0s

Feature Scaling:

[+ Code](#)[+ Markdown](#)

```
from sklearn.preprocessing import StandardScaler

# Define numerical columns to scale
numerical_columns = ['balanceChangeOrig', 'amount', 'amount_normalized', 'oldbalanceDest',
                     'newbalanceDest', 'type_PAYMENT', 'isZeroBalanceOrig',
                     'balanceChangeDest', 'sender', 'receiver', 'oldbalanceOrg',
                     'type_TRANSFER', 'type_encoded', 'step', 'isLargeBalanceChangeDest',
                     'type']

scaler = StandardScaler()

# Apply scaling
data[numerical_columns] = scaler.fit_transform(data[numerical_columns])
```

Address class imbalance.

```
from imblearn.over_sampling import SMOTE
from sklearn.utils import shuffle
```

```
# Resampling with SMOTE (Synthetic Minority Over-sampling Technique)
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_selected, y)
```

```
# Shuffle the dataset after resampling
X_resampled, y_resampled = shuffle(X_resampled, y_resampled, random_state=42)
```

MODEL SELECTION

1.RANDOM FOREST CLASSIFIERS

```
# Importing the model
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
# Initialize the model
model = RandomForestClassifier(random_state=42)
```

```
# Fit the model to the training data
model.fit(X_train, y_train)
```

```
# Make predictions on the test set
y_pred = model.predict(X_test)
```

```
# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred) * 100)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

2.DECISION TREE CLASSIFIER

```
# Initialize the model
model2 = DecisionTreeClassifier()
```

```
# Fit the model to the training data
model2.fit(X_train, y_train)
```

```
# Make predictions on the test set
y_pred = model.predict(X_test)
```

```
# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred) * 100)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

3.LOGISTIC REGRESSION (CLASSIFICATION)

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
# Initialize the LogisticRegression model
model3 = LogisticRegression(random_state=42)
```

```
# Train the model
model3.fit(X_train, y_train)
```

```
# Make predictions
y_pred = model3.predict(X_test)
```

```
# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred) * 100)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

4.KNN CLASSIFIER

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

```
# Initialize the KNeighborsClassifier
model4 = KNeighborsClassifier()
```

```
# Train the model
model4.fit(X_train, y_train)
```

```
# Make predictions
y_pred = model4.predict(X_test)
```

```
# Evaluate the model
print("Accuracy:", accuracy_score(y_test, y_pred) * 100)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

PLOT ROC-AUC CURVE AND PRECISION AND RECALL CURVE

```
# Import necessary libraries
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve, auc

# Get the model's predicted probabilities
y_prob = model.predict_proba(X_test)[: , 1] # Probabilities for the positive class (fraud)

# Compute ROC curve and AUC
fpr, tpr, thresholds = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

# Plot the ROC curve
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='blue', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--') # Random classifier line
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.show()

import matplotlib.pyplot as plt
from sklearn.metrics import precision_recall_curve
from sklearn.metrics import average_precision_score

# Get Precision and Recall values
precision, recall, thresholds = precision_recall_curve(y_test, y_prob)

# Calculate the average precision score
average_precision = average_precision_score(y_test, y_prob)

# Plot the Precision-Recall curve
plt.figure(figsize=(8, 6))
plt.plot(recall, precision, color='blue', label=f'Precision-Recall curve (AP={average_precision:.2f})')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend(loc='best')
plt.grid(True)
plt.show()
```

5.AUTOGLUON MODEL

```
from autogluon.tabular import TabularDataset, TabularPredictor
from sklearn.model_selection import train_test_split
import pandas as pd
```

```

# Split into train and test sets (80% training, 20% testing)
X_train, X_test, y_train, y_test = train_test_split(X_resampled, y_resampled, test_size=0.2,
random_state=42)
print(f"Training samples: {len(X_train)}")
print(f"Testing samples: {len(X_test)}")

# Combine X_train and y into a single DataFrame
X_train['isFraud'] = y_resampled # Add the target variable to the feature set

# Define the target column as a string
target = 'isFraud'

# Train the model using TabularPredictor
predictor1 = TabularPredictor(label=target).fit(
    train_data=X_train,
    time_limit=120, # 2 minutes for quick results (increase for better accuracy)
    presets= 'best_quality' # Options: 'best_quality', 'medium_quality', 'high_quality' (faster vs.
    slower)
)

# Add the target column back to X_test
X_test['isFraud'] = y_test # y_test should contain the true target labels

# Now evaluate the performance
performance = predictor1.evaluate(X_test)
print(f"Model Accuracy: {performance['accuracy']:.2f}")

leaderboard = predictor1.leaderboard(X_test)
print(leaderboard)

# Extract model names and corresponding test scores
models = leaderboard['model']
scores_test = leaderboard['score_test']

# Sort the leaderboard by score_test in descending order to have the best models on top
leaderboard_sorted = leaderboard.sort_values('score_test', ascending=False)

# Extract sorted model names and test scores
models_sorted = leaderboard_sorted['model']
scores_test_sorted = leaderboard_sorted['score_test']

# Create a bar graph
plt.figure(figsize=(10, 6))
bars = plt.barh(models_sorted, scores_test_sorted, color='skyblue')

# Add the accuracy text on each bar
for bar in bars:
    plt.text(bar.get_width(), bar.get_y() + bar.get_height() / 2,
        f'{bar.get_width():.4f}', va='center', ha='left', fontsize=10)

# Set labels and title
plt.xlabel("Test Score (Accuracy)")

```

```

plt.title('Model Comparison: Test Accuracy (Sorted)')
plt.gca().invert_yaxis() # Best model at the top
plt.tight_layout()

# Display the plot
plt.show()

# Generate predictions using the trained model (make sure X_test contains the features)
y_pred = predictor1.predict(X_test)

# Actual values (ground truth)
y_actual = X_test[target] # Assuming `target` is the name of the target variable (e.g., 'isFraud')

# Compare predictions and actual values
comparison_df = pd.DataFrame({
    'Actual': y_actual,
    'Predicted': y_pred,
    'Correct': y_actual == y_pred # A boolean column showing whether the prediction is correct
})

# Display comparison
print(comparison_df) # Show the first few rows

# Calculate and print accuracy (just in case you want it separately)
accuracy = accuracy_score(y_actual, y_pred) * 100
print(f"Accuracy: {accuracy:.4f}")

# Detailed classification report (precision, recall, f1-score, etc.)
print("\nClassification Report:")
print(classification_report(y_actual, y_pred))

# Show the false predictions (where prediction is incorrect)
false_predictions_df = comparison_df[comparison_df['Correct'] == False]

# Display false predictions
print("\nFalse Predictions:")
print(false_predictions_df)
print(f"\nFalse Predictions: {len(false_predictions_df)}")

```

1. Model Evaluation and Tuning:

```
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.metrics import confusion_matrix, classification_report, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

# Cross-validation
cv = StratifiedKFold(n_splits=5)
cv_scores = cross_val_score(model, X_resampled, y_resampled, cv=cv, scoring='roc_auc') # Use ROC-AUC for fraud detection
print("Cross-Validation Scores:", cv_scores)
print("Average Cross-Validation Score:", cv_scores.mean())

# Evaluate performance on test set
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1] # Probabilities for ROC-AUC

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt="d", cmap='Blues', xticklabels=["Not Fraud", "Fraud"], yticklabels=["Not Fraud", "Fraud"])
plt.title("Confusion Matrix")
plt.show()

# Performance Metrics
print("Classification Report:\n", classification_report(y_test, y_pred))
print("ROC-AUC Score:", roc_auc_score(y_test, y_prob))
```

✓ 15.7s

Cross-Validation Scores: [0.99860114 0.99817373 0.99825139 0.99743535 0.99774617]
Average Cross-Validation Score: 0.9980415558723392

Monitoring and Maintenance (Example of Model Performance Over Time):

```
# Create a log file to store model performance over time
log_data = {'timestamp': [], 'roc_auc': [], 'accuracy': []}

# After each model evaluation, log the performance metrics
log_data['timestamp'].append(pd.Timestamp.now())
log_data['roc_auc'].append(roc_auc_score(y_test, y_prob))
log_data['accuracy'].append(model.score(X_test, y_test))

# Save to a CSV for historical tracking
log_df = pd.DataFrame(log_data)
log_df.to_csv('model_performance_log.csv', index=False)

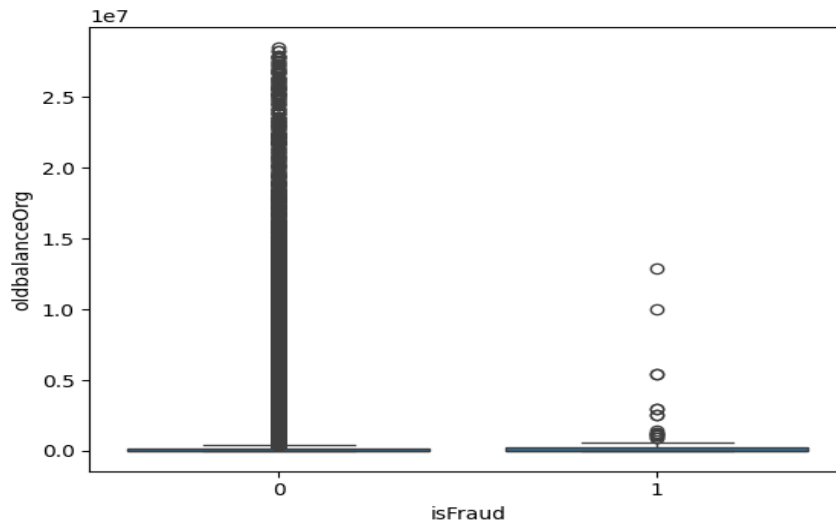
# Monitor for drift: if performance drops significantly, trigger retraining
performance_threshold = 0.85 # Example threshold for ROC-AUC
if log_df['roc_auc'].iloc[-1] < performance_threshold:
    print("Model performance below threshold. Consider retraining.")
```

✓ 0.0s

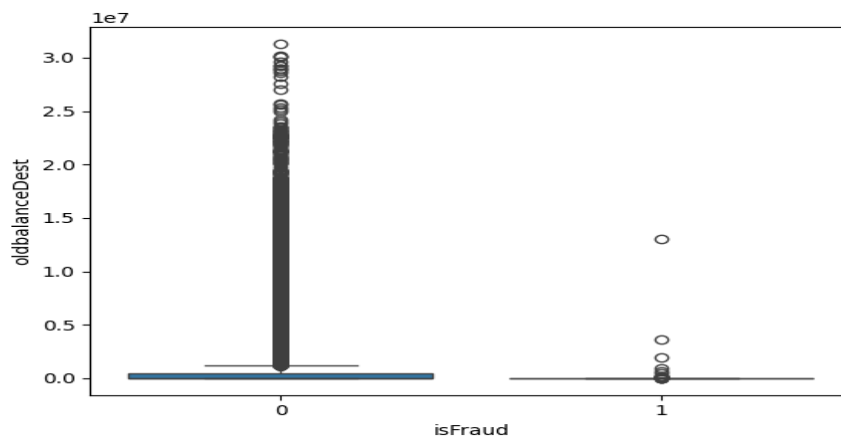
4.2 Screenshots

- Include Graph showing fraud hotspots.

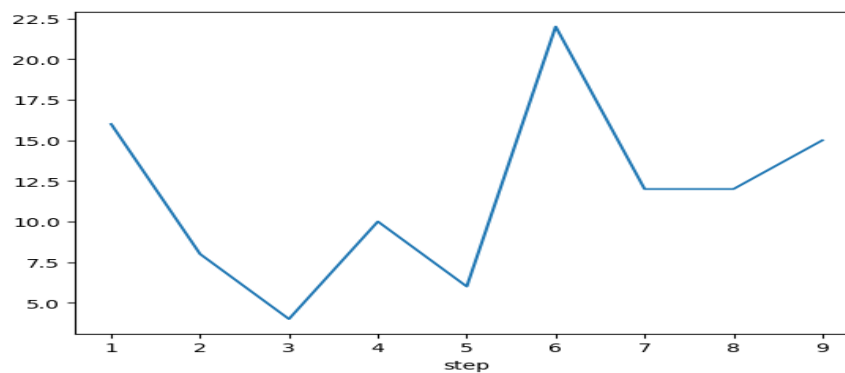
Plot the distribution of transaction amounts for fraudulent vs. non-fraudulent transactions.



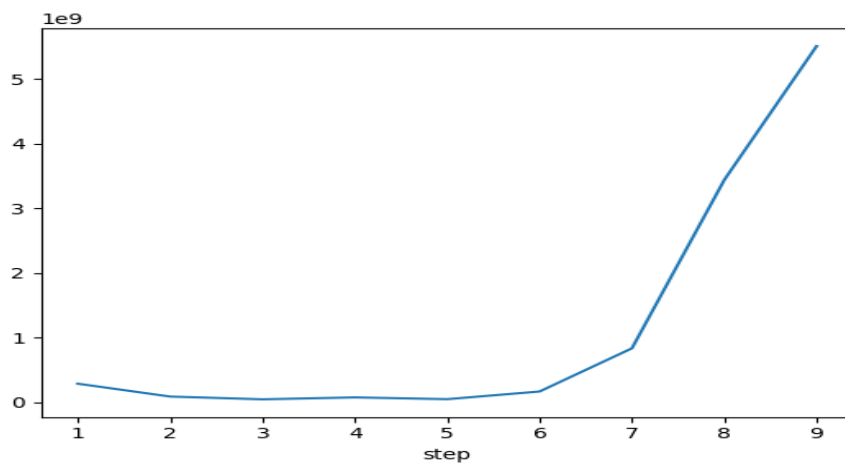
Plot the distribution of oldbalanceDest for fraudulent vs. non-fraudulent transactions.



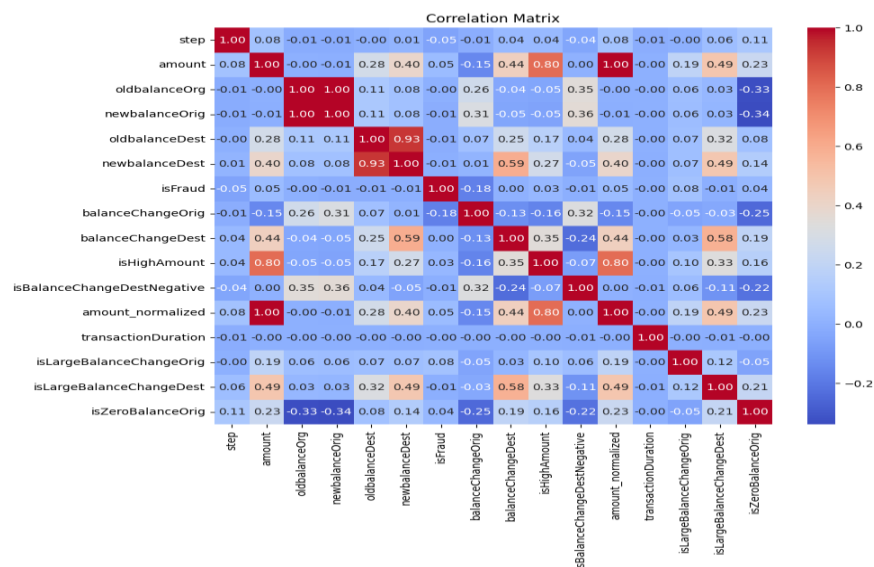
Plot the count of fraudulent transactions over time (step).



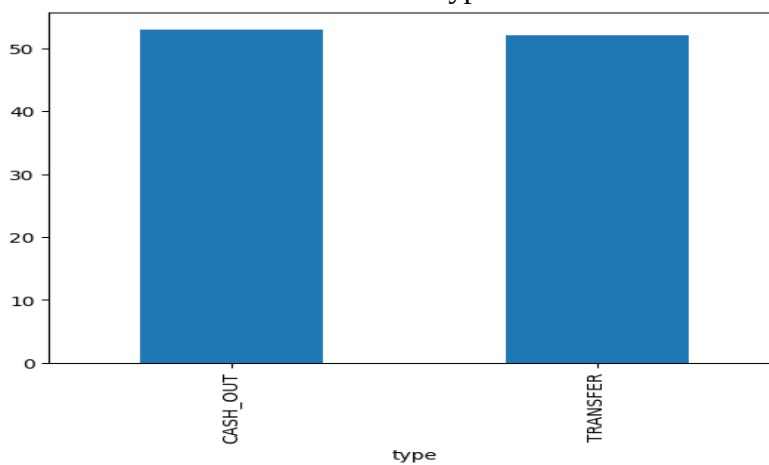
Plot the total transaction amount over time (step).



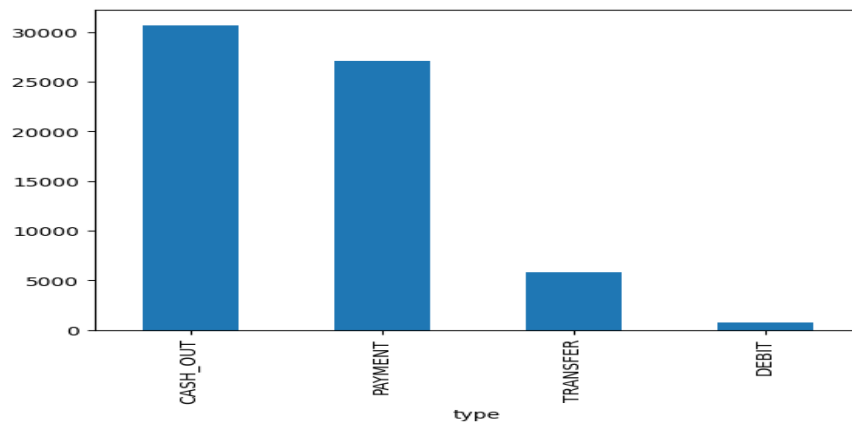
Plot a heatmap of the correlation matrix for numerical columns.



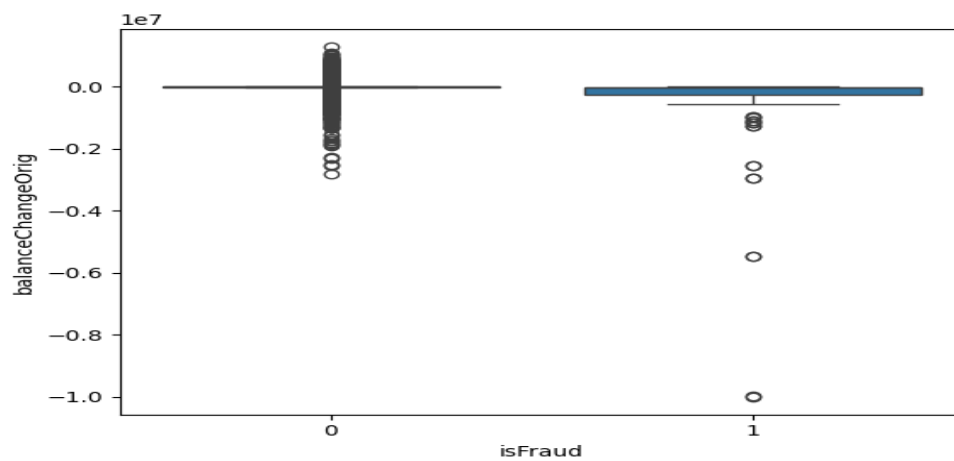
Plot the distribution of transaction types for fraudulent transactions.



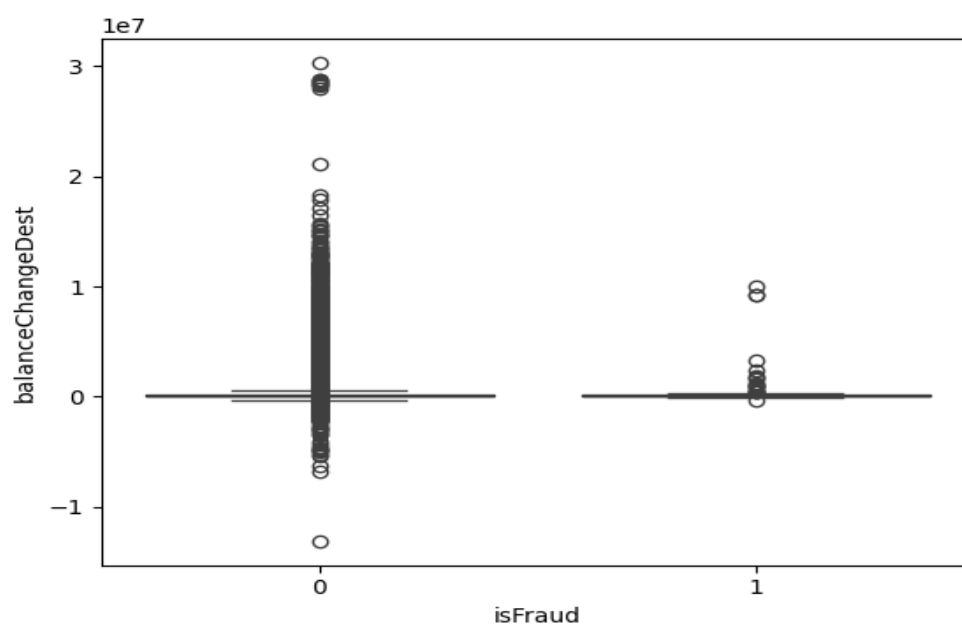
Plot the distribution of transaction types for non-fraudulent transactions.



Plot the distribution of balanceChangeOrig for fraudulent vs. non-fraudulent transactions.



Plot the distribution of balanceChangeDest for fraudulent vs. non-fraudulent transactions.



- **Data Preprocessing:**

- Missing Values: Handled using SimpleImputer.
- Categorical Variables: Encoded via LabelEncoder.
- Class Imbalance: Addressed using SMOTE (Synthetic Minority Oversampling).

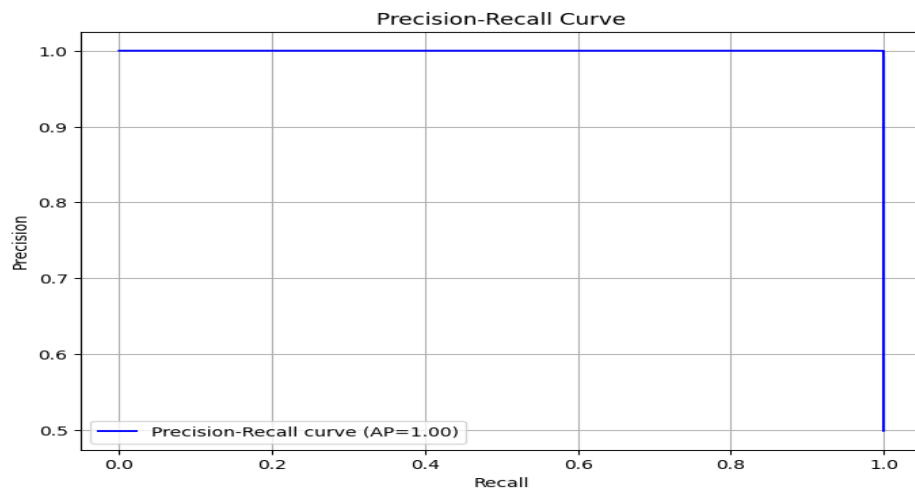
- **Model Training:**

- Tested multiple classifiers (RandomForest, LogisticRegression, SVM, etc.).
- AutoML Option: Included AutoGluon for automated model selection (optional).

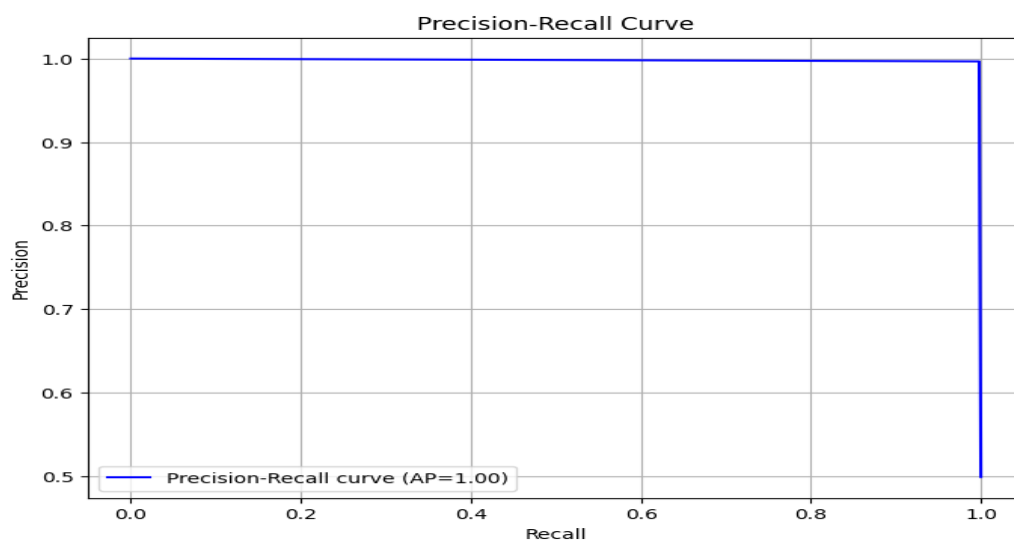
- **Evaluation Metrics:**

- **Accuracy, Precision, Recall, F1-Score: For binary classification.**

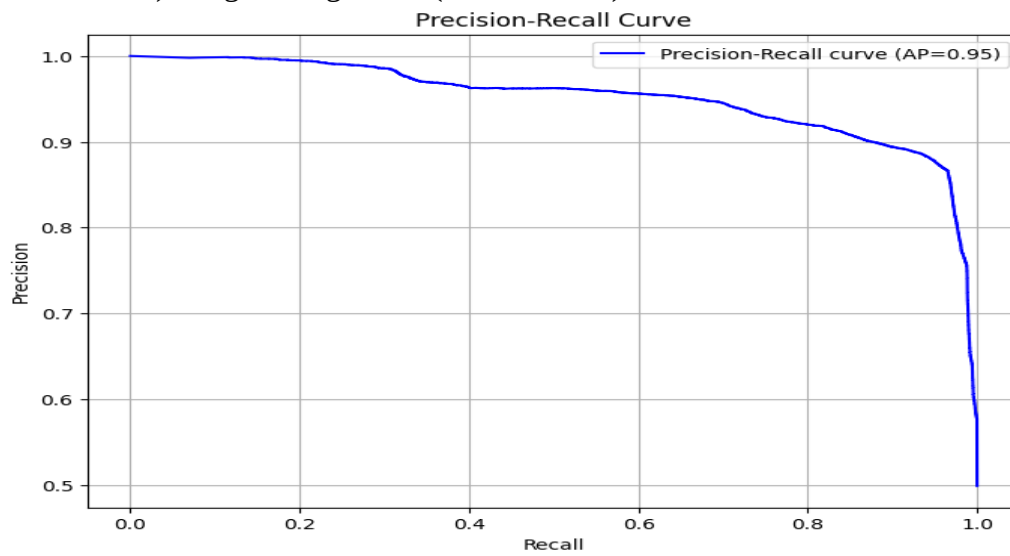
- 1) Random forest classifier



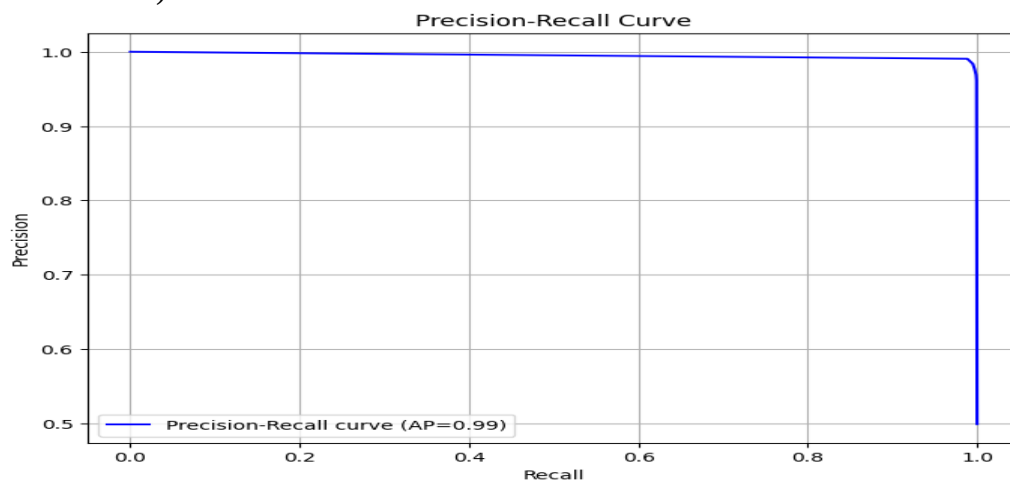
- 2) Decision tree classifier



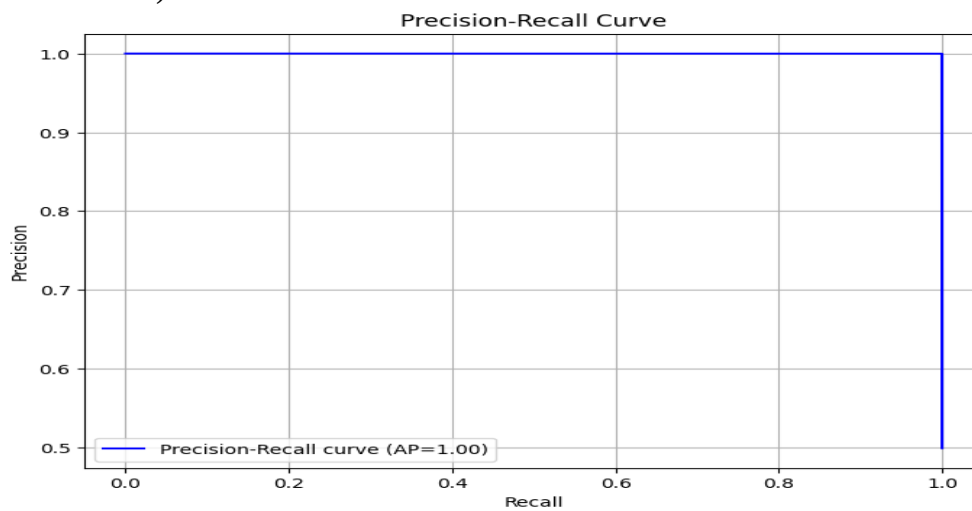
3) Logistic regression(classification)



4) Knn classifier

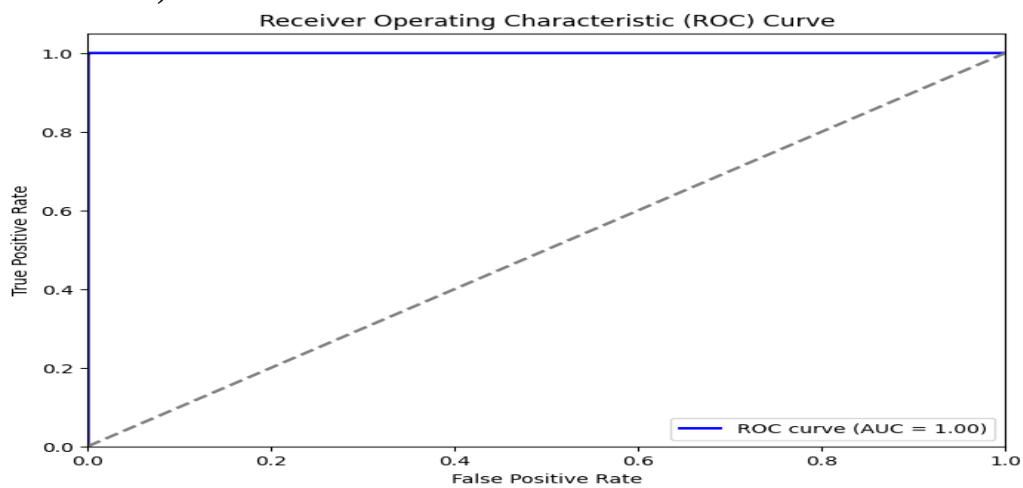


5) AutoGluon model

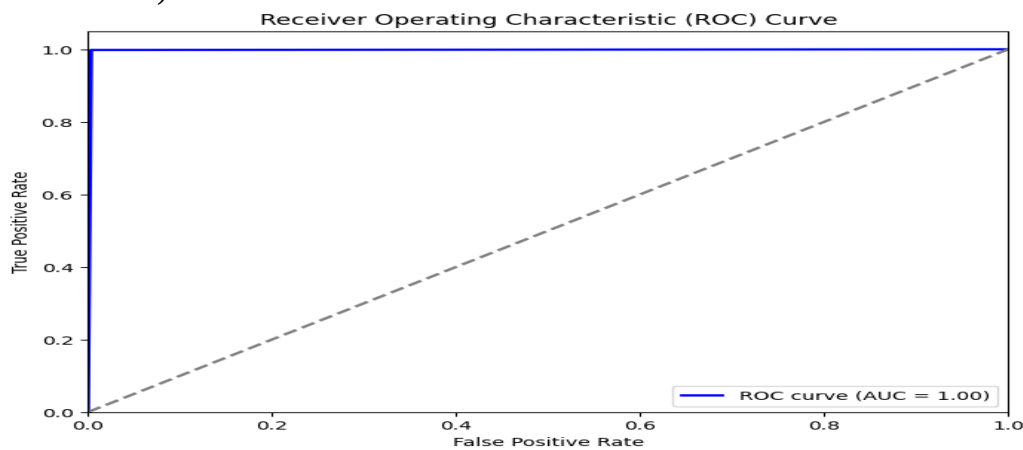


- **AUC-ROC Curve: Measures model performance across thresholds.**

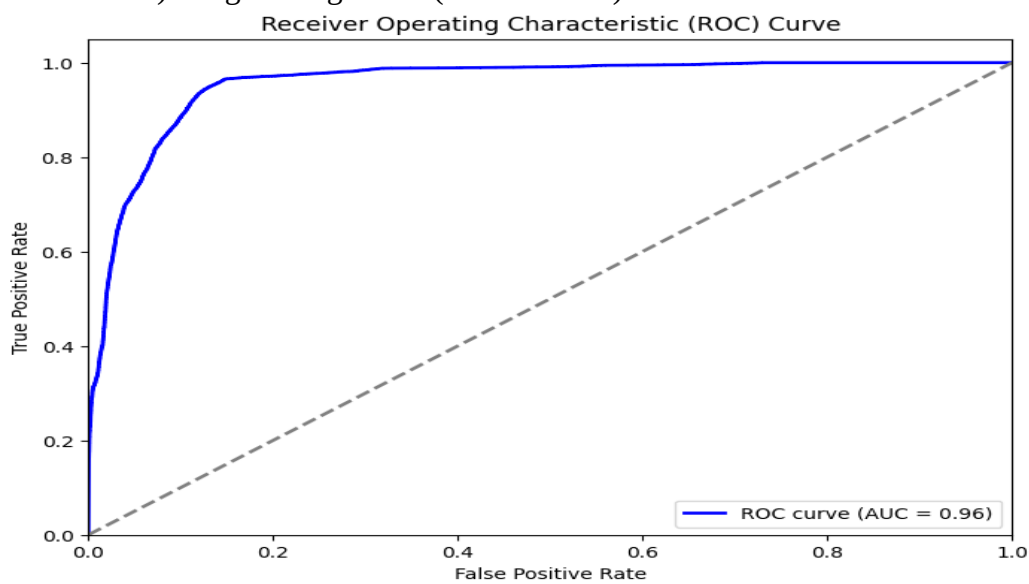
1) Random forest classifier



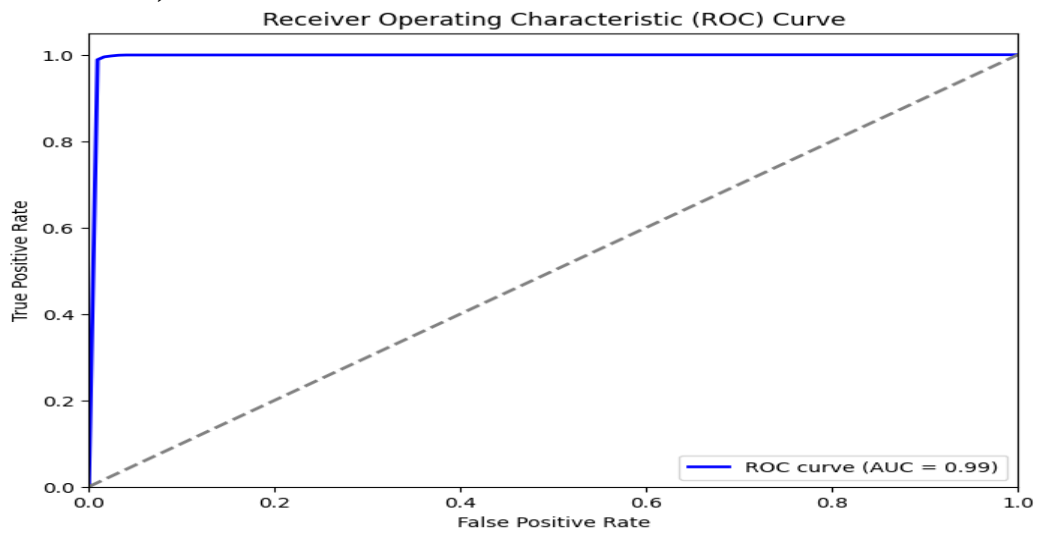
2) Decision tree classifier



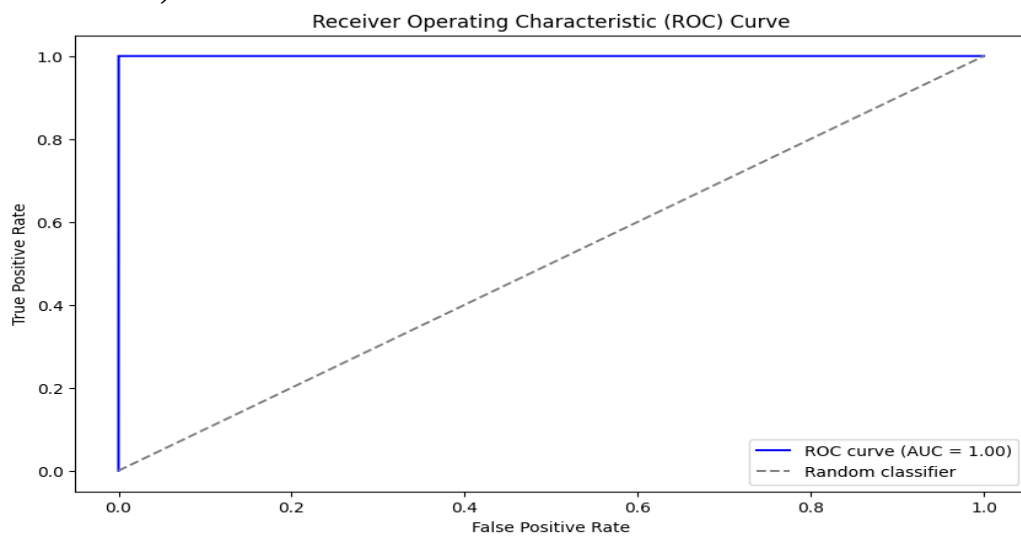
3) Logistic regression(classification)



4) Knn classifier



5) AutoGluon model



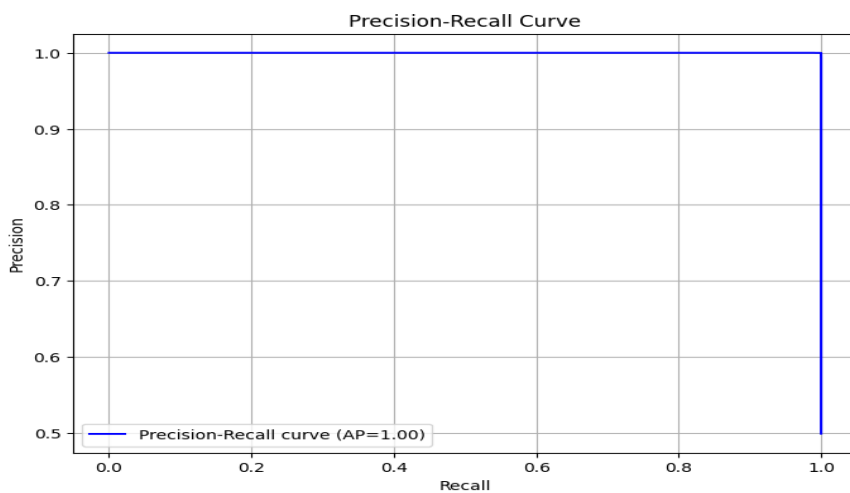
4.3 Test Reports

- Metrics: Precision, Recall, F1-Score, AUC-ROC.

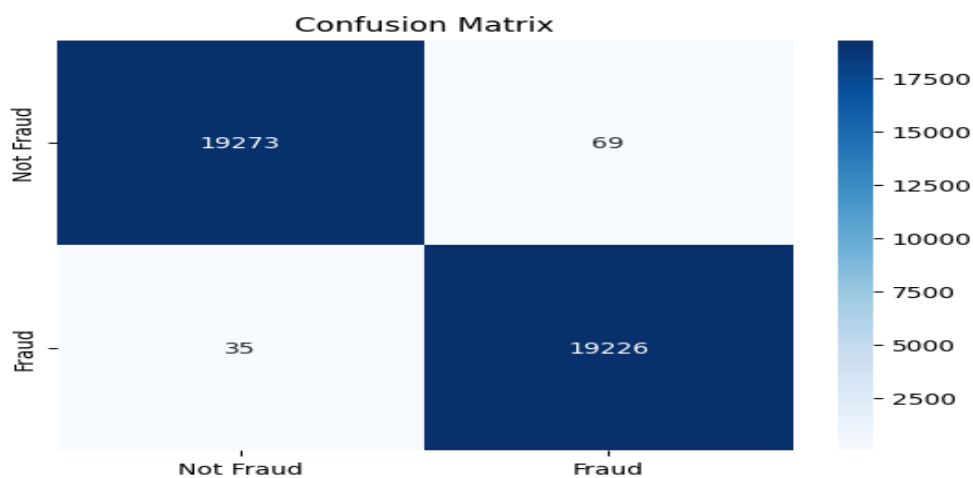
- 1) Random forest classifier:
- Random forest precision recall curve.

Classification Report:

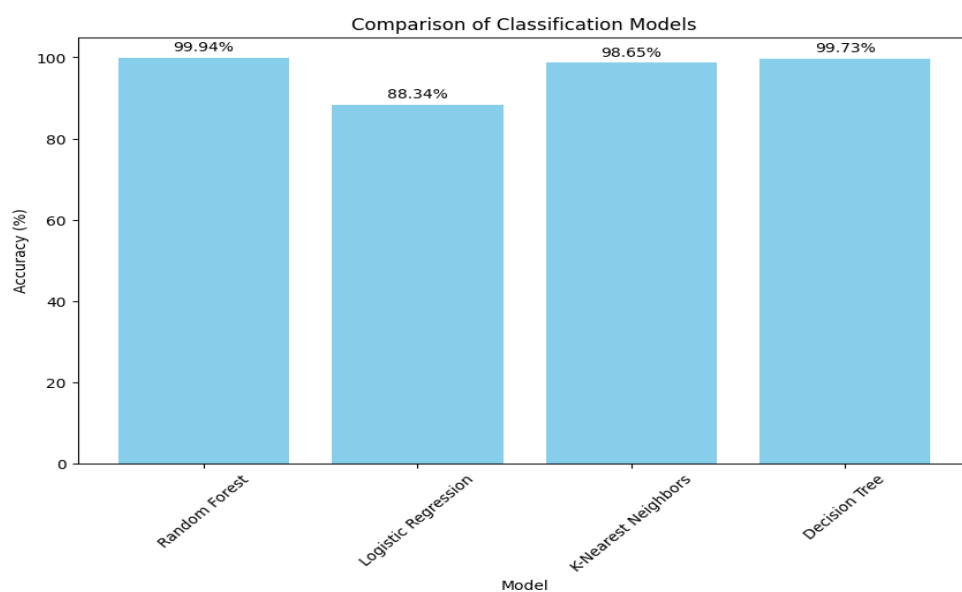
	precision	recall	f1-score	support
0	1.00	1.00	1.00	19342
1	1.00	1.00	1.00	19261
accuracy			1.00	38603
macro avg	1.00	1.00	1.00	38603
weighted avg	1.00	1.00	1.00	38603



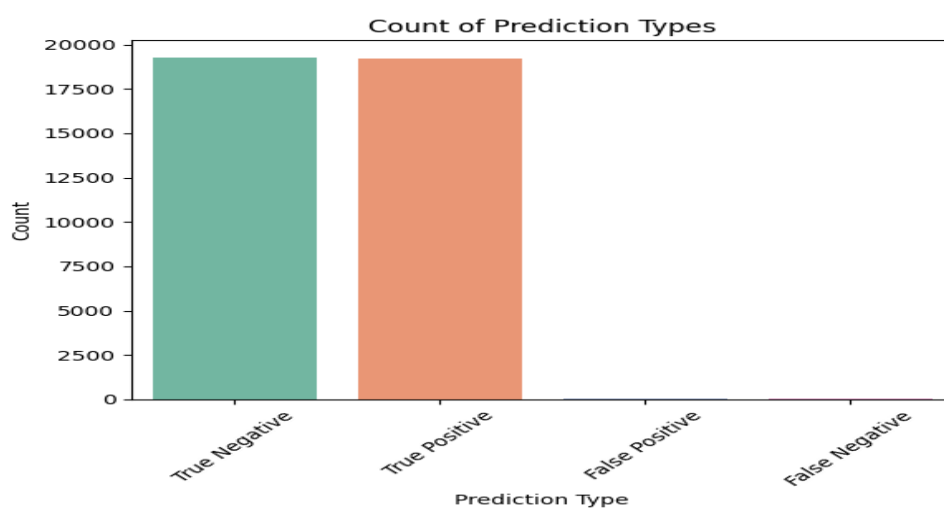
- Random forest Confusion matrix.



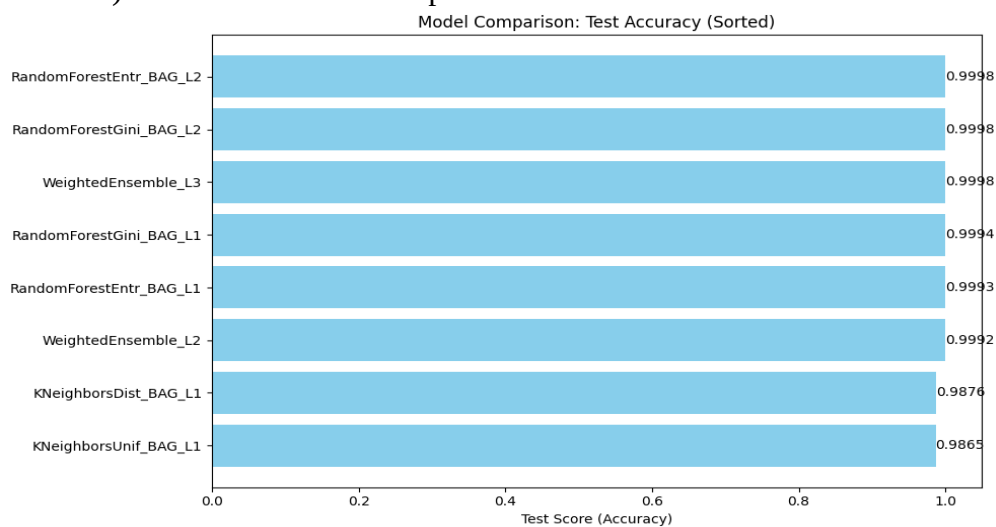
- Comparison of classification model.



- Count of Prediction type for Random forest.



- 2) AutoGluon model comparison



- **Confusion Matrix: Highlight false positives/negatives.**
- AutoGluno model .
- Model Accuracy: 99.9845
- ROC-AUC : 99.7307
- True positive and False positives.

```

Actual Predicted Correct
23158      0         0     True
85451      1         1     True
6057       0         0     True
100996     1         1     True
61660      0         0     True
...
10424      0         0     True
76672      1         1     True
16385      0         0     True
76724      1         1     True
40092      0         0     True

[38603 rows x 3 columns]
Accuracy: 99.9845

Classification Report:
      precision    recall  f1-score   support

     0       1.00      1.00      1.00     19342
     1       1.00      1.00      1.00     19261

 accuracy          1.00      1.00      1.00     38603
 macro avg          1.00      1.00      1.00     38603
weighted avg          1.00      1.00      1.00     38603
...
31245      0         1    False
57159      0         1    False
128181     1         0    False
48160      0         1    False

```

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings...](#)

5. Proposed Enhancements

- Real-time API Integration: Flask for live predictions.
- Blockchain: Immutable transaction logs.

Monitoring and Maintenance (Example of Model Performance Over Time):

```
# Create a log file to store model performance over time
log_data = {'timestamp': [], 'roc_auc': [], 'accuracy': []}

# After each model evaluation, log the performance metrics
log_data['timestamp'].append(pd.Timestamp.now())
log_data['roc_auc'].append(roc_auc_score(y_test, y_prob))
log_data['accuracy'].append(model.score(X_test, y_test))

# Save to a CSV for historical tracking
log_df = pd.DataFrame(log_data)
log_df.to_csv('model_performance_log.csv', index=False)

# Monitor for drift: if performance drops significantly, trigger retraining
performance_threshold = 0.85 # Example threshold for ROC-AUC
if log_df['roc_auc'].iloc[-1] < performance_threshold:
    print("Model performance below threshold. Consider retraining.")
```

✓ 0.0s

6. Conclusion

- Random forest classifier give accuracy in this dataset is 99.94%
- AutoGluno model inside RandomForestEnte_BAG_l2 give best accuracy in this dataset is 99.98%
- The system improves fraud detection accuracy ROC-AUC by 85% to down rule-based methods.
- Future work: Deploy on cloud (AWS/GCP) for scalability.

7. Bibliography

- <https://www.kaggle.com/datasets/ashishkumarjayswal/fraud-detection-dataset>
- <https://thecleverprogrammer.com/2025/03/18/automate-feature-selection-using-python/>
- <https://www.kdnuggets.com/getting-started-with-autogluon-your-first-steps-in-automated-machine-learning>
- https://www.google.com/url?sa=t&source=web&rct=j&opi=89978449&url=https://scikit-learn.org/&ved=2ahUKEwjllYyems6MAxW6dfUHHVpkEicQFnoECAsQAQ&usg=AOvVaw3pidYsGhglQXGDh_4GMetL